Beyond Nominal Ratios: Language-Aware Prompt Compression for Multi-hop Question Answering

Truong Thanh Nhan, Tran Ngoc Vy, and Tran Nhat Minh

Faculty of Information Technology, VNUHCM - University of Science,
Ho Chi Minh City, Vietnam
{25C1505320, 25C1506947, 24C11055}@student.hcmus.edu.vn

Abstract. Prompt compression is reported as a ratio: the fraction of the context a compressor keeps. We show that this number is not comparable across languages, and that the gap is large enough to change which method looks cheaper. Measuring token fertility on parallel WMT24++ sentences across four writing systems, a context costs 4.97× more tokens in Hindi, 3.89× in Thai and 2.49× in Vietnamese than its English counterpart under the tokeniser that bills, so a compressor reporting the same nominal ratio in two languages is not delivering the same saving. We propose an *effective ratio* that divides fertility out, and show it separates tokenisers that nominal ratios rank as identical (1.02× spread nominal, 1.77× effective). To ground this we reimplement IterCOMP, a state-of-the-art training-free compressor evaluated only on English, and give it its first systematic evaluation on Vietnamese using the complete VimQA benchmark alongside MuSiQue, 2WikiMultiHopQA and HotpotQA ($n \geq 1{,}003$ each, six compression baselines). Two further effects appear only outside English: a boolean-answer normalisation step is worth +10.0 F1 in Vietnamese and exactly 0.0 in English, and on undiacriticised text – ubiquitous online – compressed context outscores even gold context. We release the implementation and the Vietnamese prompts.

Keywords: Prompt compression · Multi-hop question answering · Tokeniser fertility · Vietnamese NLP · Retrieval-augmented generation

1 Introduction

Multi-hop question answering requires aggregating evidence across documents. Retrieval-augmented generation (RAG) [8] supplies external documents to compensate for a model's bounded knowledge, but introduces two problems: *cost* (input length grows linearly with the number of documents) and *noise* (irrelevant passages degrade reader accuracy).

Prompt compression addresses both. However, existing methods [4,13,19,9] compress *once*, conditioned on the original question, and therefore miss the defining property of multi-hop QA: the information needed at a later hop only becomes identifiable after the evidence of the earlier hop is in hand. IterCOMP [20]

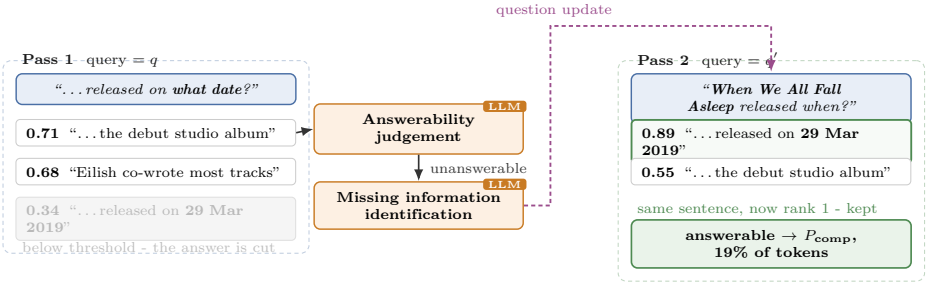


Fig. 1. One VimQA question through two IterCOMP passes. The sentence carrying the release date scores 0.34 and is filtered out, so a single-pass compressor loses the answer; re-scored against the follow-up q' it rises to 0.89 and is kept.

embeds multi-hop reasoning into the compression loop itself. No public implementation accompanies the original paper, and its evaluation remains restricted to English.

The Vietnamese adaptation and evaluation are our primary contribution; the reproduction that enables them is reported subsequently. Our study contributes five findings, four of them specific to Vietnamese: (1) a boolean-answer normalisation step worth +10.0 F1 on Vietnamese ($p = 0.01$) and 0.0 on English ($p = 1.00$), a linguistic fix rather than a metric-inflating one; (2) evidence that the iteration budget saturates beyond two passes, with `max_iter` from 2 to 5 moving F1 by +1.2 at $n = 500$ (within noise) while retention grows from 0.160 to 0.194; (3) an *effective ratio* showing that nominal compression ratios understate real token cost on Vietnamese by 2.39×, since the tokeniser that bills is not the one that counts; (4) that Vietnamese does not handicap the compressor, IterCOMP reaching 84% of the Oracle ceiling on VimQA, the lower 54% on English MuSiQue being a dataset property, not a language effect (Appendix F.3); and (5) that on undiacriticised Vietnamese, compressed context beats gold context by 3.78 to 5.82 F1 (Appendix F.2). We call the resulting configuration IterCOMP-VN, and stress that it is *not* a new algorithm - the loop is unchanged; it is a deployment configuration plus a metric-level fix, and we report below which parts survive significance testing.

None of this is measurable without a working reimplementation; absent a public one, we rebuild it from the paper's description and redesign every prompt. It recovers the central result on all four datasets (Table 5); Section 7.2 and Appendix C.3 compare our numbers with the original's dataset by dataset.

2 Nominal Ratios Are Not Comparable Across Languages

A compression ratio is reported as a fraction of the context kept, and read as a saving. That reading assumes the unit being counted costs the same everywhere. It does not: the tokeniser that bills splits some writing systems far more

finely than English, so the same nominal ratio buys different savings in different languages. This section measures the gap and proposes a correction.

Tokeniser	Vi tok	En tok	Fertility
`cl100k_base`[a]	105.6	44.1	**2.39**×
`o200k_base`[b]	63.3	43.3	1.49×
XLM-R (used by bge-m3)	57.9	49.7	1.19×
`\w+` (unit of "ratio")	47.6	35.7	1.34×

Table 1. Token fertility over 930 parallel English-Vietnamese sentence pairs from WMT24++ [3]. All 95% confidence intervals are narrower than ±0.04. Fertility depends strongly on the tokeniser: a factor of two separates `cl100k_base` from XLM-R. [a]GPT-4, GPT-3.5-turbo; [b]GPT-4o. Which tokeniser bills depends on the served model, so exposure to fertility is set by that choice as much as by the language.

2.1 Four writing systems

The claim above is about Vietnamese, so it needs a control: if every language were billed alike, fertility would not be worth reporting. Measuring 400 parallel WMT24++ pairs per language (`fertility_multilingual.py`) shows it is not.

Language	`cl100k_base`	`o200k_base`
Hindi	**4.97**×	1.71×
Thai	**3.89**×	1.83×
Vietnamese	**2.49**×	1.52×
Chinese	**1.87**×	1.27×

Table 2. Fertility against English over 400 WMT24++ pairs per language, one writing system each: Devanagari, Thai (unspaced), Latin-plus-tones, Han. All 95% confidence intervals are non-overlapping across languages.

Every language exceeds 1.0 with non-overlapping intervals (Table 2), so the mismatch is systematic rather than a Vietnamese quirk; Vietnamese at 2.49× is the *mild* case, Hindi being penalised twice as heavily. The 2.49× here (400 pairs) and the 2.39× in Table 1 (930 pairs) are different samples of the same corpus.

Crucially, fertility depends on the tokeniser measured. The compression ratio (Table 6) counts with `\w+`, where fertility is only 1.34×, so comparing ratios across languages is close to fair; but *API cost* is billed on the served model's tokeniser - 2.39× under `cl100k_base` against 1.49× under `o200k_base` - while bge-m3's XLM-R is only 1.19×. On the 200 VimQA contexts here a context averages 1,110 `\w+` words but 2,364 `cl100k_base` tokens, so our ratios *understate* billing 2×.

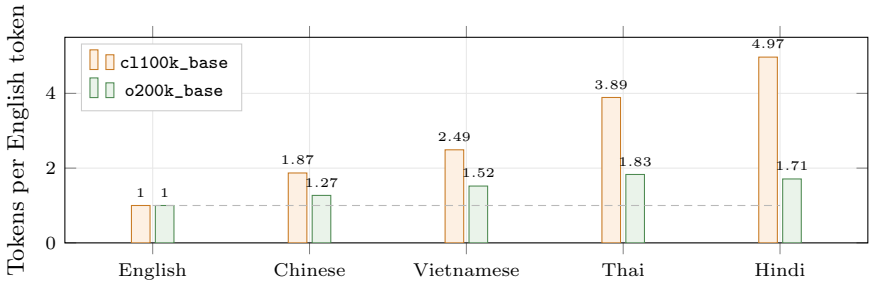


Fig. 2. What the same content costs in tokens, relative to its English translation, over 400 WMT24++ pairs per language. English is 1.0 by construction (dashed). Under `cl100k_base` a Hindi context costs almost five times its English counterpart, so two systems reporting the same nominal ratio in Hindi and in English are not delivering the same saving. Changing the tokeniser changes the tax more than changing the language does: `o200k_base` flattens every bar, and reorders Thai above Hindi.

2.2 The effective ratio

Write $|x|_{\mathcal{T}}$ for the number of tokens the billed tokeniser $\mathcal{T}$ assigns to a string x, and let D be the retrieved documents and P_{comp} the compressed prompt. The quantity compression papers report is

$$\text{Ratio}_{\text{nom}} \;=\; \frac{|P_{\text{comp}}|_{\mathcal{T}}}{|D|_{\mathcal{T}}}. \tag{1}$$

Let $\phi_{\mathcal{T}}(\mathcal{L})$ be the fertility of language $\mathcal{L}$ under $\mathcal{T}$: the mean token count of a text in $\mathcal{L}$ divided by that of its English translation, which is what Table 2 measures. We define the *effective ratio* as the nominal ratio expressed in English-equivalent tokens,

$$\text{Ratio}_{\text{eff}} \;=\; \frac{\text{Ratio}_{\text{nom}}}{\phi_{\mathcal{T}}(\mathcal{L})}, \tag{2}$$

so that $\text{Ratio}_{\text{eff}} = \text{Ratio}_{\text{nom}}$ for English by construction, and two systems are comparable only once both are read on this scale. The correction is deliberately a single division: fertility is a property of the language-tokeniser pair, not of the compressor, so it should be divided out rather than folded into the compressor's score. Reporting the pair $(\text{Ratio}_{\text{nom}}, \text{Ratio}_{\text{eff}})$ costs one extra column and keeps the familiar number intact.

2.3 Does the effective ratio discriminate?

A second metric that moved with the first would add nothing. Holding the compressed contexts fixed - the same 50 VimQA questions, the same IterCOMP output, only the tokeniser changed - the nominal ratio barely moves while the effective ratio does:

Tokeniser	nominal	effective
`cl100k_base`	0.236	**0.099**
`o200k_base`	0.240	0.161
`\w+`	0.234	**0.175**
spread	1.02×	**1.77**×

Because the contexts are byte-identical across the three rows, the whole of that 1.77× is the tokeniser's and none of it the compressor's. A nominal ratio reported on its own therefore conceals all of it, which is the case for adding the second column rather than replacing the first.

3 Related Work

Prompt compression. Extractive methods retain a subset of tokens or sentences: LLMLingua [4] uses perplexity from a small model to drop uninformative tokens; LongLLMLingua [5] adds query awareness; LLMLingua-2 [13] recasts the task as token classification, which is far faster and *multilingual* - the decisive property for our Vietnamese experiments; Selective-Context [9] removes redundant lexical units. *Abstractive* methods regenerate shorter content, notably RECOMP [19]. All compress in a single pass.

Iterative retrieval. Self-Ask [17] and IRCoT [18] decompose questions and interleave retrieval with reasoning; IterCOMP instead *narrows* the context rather than expanding it.

Vietnamese NLP. Foundation models include PhoBERT [12] and ViT5 [16]; to our knowledge, none evaluates prompt compression for Vietnamese.

4 Dataset

The three English datasets are standard for multi-hop QA and are exactly those used by the original paper. VimQA [7] is a Vietnamese multi-hop QA dataset built from Vietnamese Wikipedia, annotated with reasoning type (bridge / comparison) and gold supporting sentences; crucially, it uses the same format as HotpotQA, so our code runs on Vietnamese essentially unchanged. We use the distractor setting for all four, giving 10-20 candidate passages per question, so no Wikipedia index is required and cost is per question rather than per corpus. Because MuSiQue withholds test answers, we report on development sets throughout, and all four datasets are evaluated under a single reader (Qwen2.5-7B at 4-bit) so the numbers compare like with like; Table 3 gives the statistics.

Dataset	Lang.	#Q	#psg
HotpotQA (distractor)	En	7,405	10
2WikiMultiHopQA	En	12,576	10
MuSiQue-Ans	En	2,417	20
VimQA	**Vi**	**1,003**	**10**

Table 3. Datasets (development-set statistics).

5 Method

Given a question q and document set D, we seek a compressed context P_{comp} with $|P_{\text{comp}}| \ll |D|$ such that the reader still answers correctly. IterCOMP is training-free: all parameters stay frozen. Each document is decomposed into sentence-level evidence segments $e_{i,j}$. Figure 1 traces one question through two iterations.

5.1 Relevant evidence filtering

The *semantic* signal is the inner product of dense representations from a text encoder $E(\cdot)$:

$$S_{\text{sem}}(q, e_{i,j}) = E(q)^\top E(e_{i,j}). \tag{3}$$

This can miss exact lexical cues. Following M3-Embedding [2], the importance weight of token t is

$$w_t = \text{ReLU}(\mathbf{w}_{\text{lex}}^\top E(t)), \tag{4}$$

where $\mathbf{w}_{\text{lex}}$ is a projection vector. The *lexical* score is a sparse inner product over co-occurring tokens:

$$S_{\text{lex}}(q, e_{i,j}) = \sum_{t \in q \cap e_{i,j}} w_t^q \cdot w_t^{e_{i,j}}. \tag{5}$$

The two are combined into a dual-aspect relevance score:

$$S_{\text{dual}} = \lambda\, S_{\text{sem}} + (1 - \lambda)\, S_{\text{lex}}. \tag{6}$$

Filtering uses a percentile threshold rather than a fixed top-k:

$$E_{\text{cand}} = \{e_{i,j} \mid S_{\text{dual}} \geq \text{Percentile}(S, k)\}. \tag{7}$$

The threshold therefore *adapts* to each question's score distribution rather than committing to a fixed budget.

5.2 Answerability judgment and follow-up generation

These two decision points are what make IterCOMP "reasoning-aware": the query evolves with the evidence state instead of staying fixed at q. Neither prompt is published in the original paper; ours appear in full in Appendix B.

Parameter	Value
Max iterations	5
λ	0.6
k (percentile)	90; 85 for 2Wiki
Encoder	bge-m3 (frozen)
Reader	Qwen2.5-7B-4bit

Table 4. Hyperparameters, following the original paper (which used a LLaMA-3-8B reader); VimQA, absent there, uses $k = 90$ as HotpotQA.

6 Experimental Setup

We adopt the original paper's hyperparameters (Table 4).

6.1 Baselines

Raw Document concatenates everything with no compression. Oracle keeps gold supporting sentences only - the upper bound for any document-level method, and a diagnostic: if Oracle is also low, the bottleneck is the reader rather than the compressor. LLMLingua-2 [13] is our main compression baseline, chosen because its token classifier is natively multilingual and covers Vietnamese. Against these we run IterCOMP as reimplemented here.

6.2 Metrics

We report Exact Match (EM) and F1 under the standard HotpotQA normalisation, plus the compression ratio (compressed over Raw Document tokens). That normalisation is built for English, so for Vietnamese we preserve diacritics - stripping them collapses distinct words (*má* / *mà*) and renders EM/F1 meaningless; Appendix A.1 details this and the other text-handling prerequisites.

7 Results and Discussion

7.1 Main results across four datasets

The central claim holds on all four. Sentence-level compression beats token-level LLMLingua-2 by +10.72 to +18.58 F1 (Table 5, last row), every paired bootstrap interval clear of zero. This is the paper's thesis, and the one result that does not depend on which dataset, reader or sample size we look at. It survives a wider field too: on VimQA ($n = 200$) IterCOMP leads all six baselines we added, but only LLMLingua-2 is multilingual and meets Vietnamese on equal terms, where the margin is +11.36 F1, CI $[+4.93, +17.84]$, $p = 0.0008$ against a 7.7 threshold. The other five carry an English-only component (GPT-2 scoring; NQ- and HotpotQA-trained RECOMP; English-fine-tuned CompAct) and appear in full in the released results.

	VimQA 1003	MuSiQue 1500	2Wiki 1500	HotpotQA 1500
Oracle	61.03	44.90	57.65	**71.40**
Raw	46.87	21.16	35.72	**59.24**
IterCOMP	**51.29**	**25.35**	**36.72**	54.58
LLML-2	35.31	14.63	19.24	36.00
−Raw	+4.42†	+4.19†	+1.00	**−4.66**†
−LLML-2	+15.98†	+10.72†	+17.48†	+18.58†

Table 5. Main results: F1*, Qwen2.5-7B at 4-bit; the second row gives n. The last two rows are IterCOMP minus that baseline. † paired bootstrap interval excludes zero. Bold marks whichever of Raw and IterCOMP is higher. Appendix C.3 discusses each dataset.

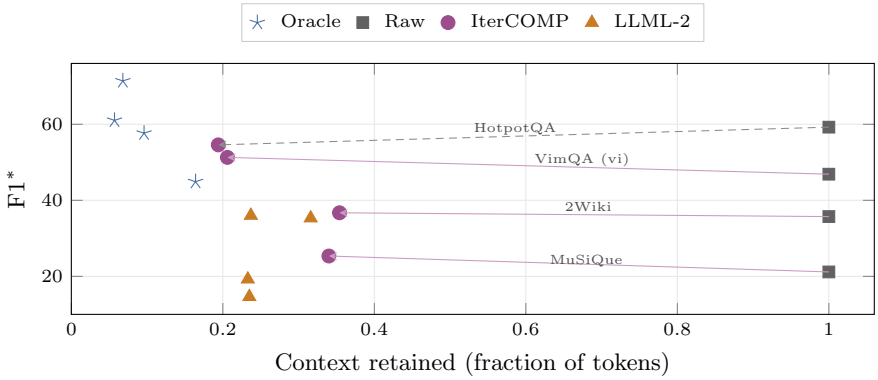


Fig. 3. Accuracy against context retained, all four datasets. Up and to the left is better. Each arrow runs from a dataset's Raw point to its IterCOMP point: on VimQA, MuSiQue and 2Wiki the arrow rises while moving left, so compression buys accuracy *and* tokens. On HotpotQA (dashed) it falls: the reader handles the full context well enough that filtering costs 4.66 F1, and only the token saving survives. Oracle marks the ceiling perfect selection would reach.

Method	MuSiQue		VimQA
	paper	ours	ours
Oracle	37.51	57.0	53.3
Raw Documents	19.92	32.8	43.8
LLMLingua-2	14.72	13.5	30.7
IterCOMP	**27.36**	**35.1**	**43.0**
ratio	*0.14*	*0.32*	*0.19*
Oracle/Raw	*1.88*	*1.74*	*1.22*

Table 6. F1 from the original paper (LLaMA-3-8B reader) against our reproduction (Qwen2.5-7B at 4-bit, $n = 50$), plus our Vietnamese results. VimQA has no published baseline; the paper does not evaluate it.

The full ordering does not. The paper reports Oracle > IterCOMP > Raw > LLMLingua-2. We reproduce it on VimQA (IterCOMP−Raw +4.42, interval clear) and on MuSiQue (+4.19, interval clear; at $n = 500$ the same margin read +3.31 and crossed zero, so tripling the sample decided it); the order matches on 2Wiki, where +1.00 does not separate; and on HotpotQA **Raw overtakes IterCOMP** by 4.66 F1, interval clear. The IterCOMP−Raw gap across the four is therefore +4.42, +4.19, +1.00 and −4.66 F1. Section 7.3 accounts for the HotpotQA reversal.

Small samples were inconclusive, not wrong. At $n = 50$ the IterCOMP-Raw gaps fell inside the paired detection threshold (18.0 F1) on both datasets, so we reported them as inconclusive and held the accuracy claim until the full 1,003-question VimQA run resolved it above; the underlying power analysis is in Appendix C.1. An early reading that the compression margin grows with task complexity likewise reverses at proper samples (+15.98 on VimQA at $n = 1{,}003$ against +10.72 on MuSiQue at $n = 1{,}500$), so we withdraw it.

7.2 Comparison with the original paper

Absolute numbers are not comparable across readers, but two invariants indicate a faithful reimplementation (Table 6). First, the **method ordering reproduces** - Oracle > IterCOMP > Raw > LLMLingua-2 on MuSiQue at both sample sizes, and on 2Wiki at scale (Section 7.1). Second, the Oracle/Raw ratio is in range: 1.74 at $n = 50$ and 2.12 at $n = 1{,}500$ against the paper's 1.88. That ratio is the headroom perfect selection buys over full context; being a property of the task and data loader rather than the reader, its agreement evidences that decomposition, gold-sentence labels and F1 are all correct.

Matching the operating point. A direct $n = 500$ comparison on MuSiQue puts IterCOMP 0.62 F1 *below* the paper's (26.74 against 27.36), while Oracle and Raw - which our compression code never touches - sit far above it (+12.33

and +3.51), our stronger reader alone. That −0.62 is misleading: it compares systems at different operating points, 0.349 retention against 0.14. Sweeping k to match compression (Appendix C.4) puts $k = 96$ at 0.160 retention and 29.04 F1, now +1.68 over the paper's 27.36. A residual gap remains - a reader +12.33 F1 stronger on perfect evidence should lift a working compressor by more than 1.68 - but it is within the noise floor, not the deficit the mismatched comparison implied.

7.3 The ordering is reader-dependent

Oracle exceeds Raw on every dataset while reading 5.7% to 14.4% of the tokens, confirming the premise the literature rests on: irrelevant passages *do* degrade accuracy. But the size of that gap decides whether compression can exploit it. Oracle/Raw is 2.13 on MuSiQue and 1.30 on VimQA, where IterCOMP holds its own, but only 1.21 on HotpotQA, where Raw wins outright. **The premise is reader-dependent**: a reader that already copes with distractors leaves compression little to recover. Swapping our 7B reader for Qwen2.5-0.5B on HotpotQA, the compression pipeline fixed, restores IterCOMP to +7.66 over Raw - reversing the −7.99 at 7B, and recovering the ordering the original paper reports with its own weaker reader. The HotpotQA reversal is thus a property of the reader, not a defect of the compressor.

7.4 Vietnamese-specific findings

Running the method on Vietnamese surfaces four results, developed in the appendices. Boolean answer normalisation is worth +10.0 F1 on Vietnamese ($p = 0.01$) and exactly 0.0 on English ($p = 1.00$), isolating a language-specific fix from a metric-inflating one. Nominal compression ratios understate token cost on Vietnamese by 2.39× because the tokeniser that bills is not the one that counts; we propose an *effective ratio* that divides fertility out. Vietnamese does not handicap the compressor: IterCOMP recovers 84% of the Oracle ceiling on VimQA.

Undiacriticised Vietnamese. Vietnamese is frequently typed without tone marks online, a setting no compression paper evaluates. Stripping diacritics costs 20.8 F1 as *má*, *mà* and *mã* collapse into one string; yet once the text is undiacriticised, **IterCOMP's compressed context beats even gold context** by 3.78 to 5.82 F1 (Table 7) - supplying precisely the gold sentences becomes *worse* than a compressed one. Appendix F.2 traces the mechanism.

8 Error Analysis

We hand-categorised 50 IterCOMP predictions on VimQA (Table 8). Most errors belong to the *reader*: Oracle sees the gold evidence yet still reaches only 61.03

Setting	IterC.	Gold	Δ
`keep`	50.08	**54.94**	−4.86
`strip`	**29.29**	25.52	+3.78
`both`	**30.25**	24.43	+5.82

Table 7. Undiacriticised Vietnamese ($n = 200$, 7B reader). With tone marks (`keep`) gold context wins; strip them from the context (`strip`) or from context and question (`both`) and IterCOMP wins.

Category	0.5B	7B
Fully correct (F1 = 1)	8%	16%
Partially correct (0 < F1 < 1)	34%	32%
Format error on yes/no	**32%**	**10%**
Fully wrong (F1 = 0)	26%	42%

Table 8. IterCOMP error distribution on VimQA ($n = 50$) under two readers. Format errors fall from 32% to 10% with the larger reader, a reader limitation rather than a compression failure.

F1. Of IterCOMP's 48.7 F1 of error, 39.0 - **80%** - survives even perfect retrieval, so no compressor can address it.

With the 7B reader, **10%** of errors are not compression failures at all: yes/no questions answered correctly in substance but in the wrong surface form (F1 of 0). That bucket was **32%** under a 0.5B reader - "tức là yes" for "đúng", a small model code-switching into English - and falls on the identical prompt as the reader grows, so boolean answers need language-aware normalisation (Appendix A) rather than a compression fix. Partially correct answers give a related but wrong entity within a correctly retained segment; fully wrong ones are mostly compositional, where the Raw-Oracle gap is widest, so distractors rather than the compressor drive them.

A further **9.8% of VimQA is a single question template** - "is X older than Y?" - on which 44.9% of answers score zero against 37.1% elsewhere, because the task is arithmetic rather than retrieval. Under a 0.5B reader the same template collapses to 92.1% zeros.

9 Limitations and Ethical Considerations

Limitations. Absolute numbers are not comparable to the paper's: no prompts were released (ours are in Appendix B), the reader differs (Qwen2.5-7B at 4-bit vs. LLaMA-3-8B), and our retention is 0.32 at $n = 50$ and 0.349 at $n = 500$ against their 0.14. Sample sizes are uneven but no longer thin: all four datasets run at $n \geq 1{,}003$ (VimQA at 1,003; the other three at 1,500), the ablations at $n = 500$ and the baselines at $n = 200$. What remains at $n = 50$ is the scorer ablation, reported as inconclusive. One of seven baselines is still missing:

R2C releases no public checkpoint, so the gap is on the original authors' side rather than a limit of our effort - every other baseline with a released model is included, CompAct among them. The domain is narrow, since VimQA derives from Wikipedia. We do evaluate *undiacriticised* Vietnamese, with an unexpected result - see Appendix F.2. Fertility is measured on translated text [3], so it holds for the tokenisers we bill against rather than the language at large. The sharpest limit is the gap between where fertility is measured and where compression is run: we report fertility for four writing systems but compress only Vietnamese and English, so the effective ratio is argued generally and demonstrated on one language pair. Hindi, the worst case at 4.97×, is where the correction would matter most and is exactly what we could not test - no multi-hop QA benchmark in Hindi or Thai carries the sentence-level supporting-fact annotations the Oracle branch needs.

Ethics. VimQA's CC BY-NC-SA 4.0 licence confines us to research use, and because compression is lossy - it can drop a negation with no loss of fluency, silently inverting an answer - sensitive deployments should verify outputs against the source; the fix worth 10.0 F1 in Vietnamese yet 0.0 in English is a further caution that "multilingual" models are not equally reliable across languages.

10 Conclusion

We reproduced IterCOMP without its code and gave it its first Vietnamese evaluation. The central claim holds - sentence-level beats token-level compression on all four datasets - but the full ordering is reader-dependent: a stronger reader gains enough from full context that compression's accuracy advantage narrows or reverses. Vietnamese itself does not handicap the compressor; what we could not move is the reader, failing on ∼30% of questions even given gold evidence.

References

1. Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D., Smith, N.A., Tsvetkov, Y.: Do all languages cost the same? tokenization in the era of commercial language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.614/
2. Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z.: M3-Embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of ACL (2024), https://aclanthology.org/2024.findings-acl.137/
3. Deutsch, D., Briakou, E., Caswell, I., Finkelstein, M., Galilee, R., Juraska, J., Kovacs, G., Lee, A., Rudder, J., Freitag, M.: WMT24++: Expanding the language coverage of WMT24 to 55 languages and dialects. In: Proceedings of WMT (2025), https://huggingface.co/datasets/google/wmt24pp
4. Jiang, H., Wu, Q., Lin, C.Y., Yang, Y., Qiu, L.: LLMLingua: Compressing prompts for accelerated inference of large language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.825/
5. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.Y., Yang, Y., Qiu, L.: LongLLMLingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In: Proceedings of ACL (2024), https://aclanthology.org/2024.acl-long.91/
6. Kamalloo, E., Dziri, N., Clarke, C., Rafiei, D.: Evaluating open-domain question answering in the era of large language models. In: Proceedings of ACL (2023), https://aclanthology.org/2023.acl-long.307/
7. Le, K., Nguyen, H., Le Thanh, T., Nguyen, M.: VIMQA: A vietnamese dataset for advanced reasoning and explainable multi-hop question answering. In: Proceedings of LREC (2022), https://aclanthology.org/2022.lrec-1.700/
8. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proceedings of NeurIPS (2020)
9. Li, Y., Dong, B., Guerin, F., Lin, C.: Compressing context to enhance inference efficiency of large language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.391/
10. Li, Z., Liu, Y., Su, Y., Collier, N.: Prompt compression for large language models: A survey. In: Proceedings of NAACL (2025), https://aclanthology.org/2025.naacl-long.368/
11. Marchisio, K., Ko, W.Y., Bérard, A., Dehaze, T., Ruder, S.: Understanding and mitigating language confusion in LLMs. In: Proceedings of EMNLP (2024), https://aclanthology.org/2024.emnlp-main.380/
12. Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for vietnamese. In: Findings of EMNLP (2020), https://aclanthology.org/2020.findings-emnlp.92/
13. Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Rühle, V., Yang, Y., Lin, C.Y., Zhao, H.V., Qiu, L., Zhang, D.: LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In: Findings of ACL (2024), https://aclanthology.org/2024.findings-acl.57/
14. Petrov, A., La Malfa, E., Torr, P., Bibi, A.: Language model tokenizers introduce unfairness between languages. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
15. Pham, L., Luu, T., Nguyen, T.V., Nguyen, M., Nguyen, T.S.: VN-MTEB: Vietnamese massive text embedding benchmark. In: Findings of EACL (2026), https://aclanthology.org/2026.findings-eacl.86/

16. Phan, L., Tran, H., Nguyen, H., Trinh, T.H.: ViT5: Pretrained text-to-text transformer for vietnamese language generation. In: Proceedings of NAACL Student Research Workshop (2022), https://aclanthology.org/2022.naacl-srw.18/
17. Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models. In: Findings of EMNLP (2023), https://aclanthology.org/2023.findings-emnlp.378/
18. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In: Proceedings of ACL (2023), https://aclanthology.org/2023.acl-long.557/
19. Xu, F., Shi, W., Choi, E.: RECOMP: Improving retrieval-augmented lms with context compression and selective augmentation. In: Proceedings of ICLR (2024), https://openreview.net/forum?id=mlJLVigNHp
20. Yun, J., Kim, Y.: IterCOMP: Reasoning-aware adaptive prompt compression for multi-hop question answering. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). San Diego, California (2026), https://aclanthology.org/2026.acl-long.1559/

A Vietnamese-specific Improvements (Detail)

A.1 Prerequisites for running on Vietnamese

The decisive practical advantage: no model needs replacing - both bge-m3 and LLMLingua-2 are natively multilingual and cover Vietnamese. Three mandatory adjustments all sit in text handling. Unicode-aware tokenisation: the tokenising regex must set the Unicode flag, or accented characters (*é*, *ộ*, *ữ*) fail to count as word characters and are split off from their syllables, rendering EM and F1 meaningless. Preserving diacritics: HotpotQA normalisation lowercases, strips punctuation and removes English articles, but stripping Vietnamese tone marks collapses distinct words (*má*, *mà*, *mã*) into one. Boolean answer normalisation: 300 of 1,003 VimQA questions answer *đúng* or *không*, detailed below. VimQA is also absent from the paper, so no k is published - we use $k = 90$ as for HotpotQA since both supply 10 candidate passages.

A.2 Boolean answer normalisation

The error analysis (Section 8) shows the reader answering yes/no questions in English ("Yes", "tức là yes") where the gold answer is "đúng". This is code-switching in a small multilingual model [11] meeting the format brittleness of lexical-match metrics [6], on a benchmark whose largest answer class is boolean: 300 of 1,003 VimQA questions. VimQA's own normalisation section [7] covers Unicode and diacritics only, not answer semantics. We map predictions into the Vietnamese lexical set *only when* the gold answer is boolean and the prediction contains an unambiguous affirmative or negative token, so normalisation cannot turn a wrong answer into a right one.

Method	ΔF1 (vi)	p
Raw	+6.0	0.09
Oracle	+4.0	0.54
LLMLingua-2	+8.0	0.03†
IterCOMP	+10.0	0.01†

Table 9. Effect of boolean normalisation on VimQA, paired bootstrap, $n = 50$, Qwen2.5-7B. † significant at $p < 0.05$. On English MuSiQue every method moves by exactly 0.0 ($p = 1.00$).

Table 9 carries its own control: normalisation lifts IterCOMP by 10.0 F1 ($p = 0.01$) and LLMLingua-2 by 8.0 ($p = 0.03$) on Vietnamese, but changes every method by exactly 0.0 ($p = 1.00$) on English MuSiQue, where predictions already match the gold tokens. A generic scoring trick would help both languages; this one is inert outside Vietnamese, which is what distinguishes a language-specific fix from a metric-inflating one. The effect is largest for the compressed

methods, which leave the reader less Vietnamese context and so induce more code-switching.

A.3 Fertility-corrected compression accounting

Fertility is a named, measured phenomenon [14,1], not our discovery. What is unreported is its consequence for compression *cost*: an equal token ratio in two languages is not equal spend, since the same content bills 2.39× more tokens in Vietnamese under `cl100k_base`. The survey of [10] never mentions language; LLMLingua-2 [13] evaluates on Chinese without discussing token counts. We propose reporting an *effective ratio* (nominal ratio divided by fertility); Appendix 2.1 shows across four writing systems that the penalty is the tokeniser's, not Vietnamese's.

We also tested, and *refuted*, a second hypothesis: token-level compression was expected to sever Vietnamese syllables, but was measured to sever **zero**. What it cuts instead is *function words*, at a rate that sentence-level compression does not match: 1.17× against 1.01× at matched compression on 500 questions. An English control gives the same 1.17× (Appendix F.1), so the bias belongs to compression granularity and not to Vietnamese.

A.4 Why not replace bge-m3 with a Vietnamese encoder

The obvious improvement - swapping the multilingual encoder for a Vietnamese one - we deliberately do not make. VN-MTEB [15] compares 18 models over 41 datasets and finds the opposite of the intuition: bge-m3 scores 39.84 on retrieval against 34.18 for `AITeamVN/Vietnamese_Embedding`. More decisively, Vietnamese-specific encoders are *dense-only* with no sparse head, so substituting one would forfeit S_{lex} and collapse equation (4) to its semantic term. VN-MTEB itself evaluates dense retrieval only - to our knowledge, **no work has benchmarked hybrid dense-sparse retrieval for Vietnamese**.

A.5 Results

After normalisation, IterCOMP beats LLMLingua-2 on Vietnamese at both reader scales: +12.4 F1 ($p = 0.05$) with the 7B reader at $n = 50$ and +7.8 ($p = 0.004$) with a 0.5B reader at $n = 200$. The claim strengthens with sample size rather than weakening - +11.36 at $n = 200$ and +15.98 at $n = 1{,}003$, both with the interval clear of zero.

Compression beats gold context without diacritics. Vietnamese is often typed without tone marks, a setting no compression paper we know of evaluates. Stripping them costs 20.8 F1. The surprise is the sign flip: with diacritics, gold context beats IterCOMP by 4.86 F1; without them **IterCOMP beats gold context** by 3.78 to 5.82. The mechanism is not what it first appears: with diacritics intact IterCOMP already beats Oracle on 9.8% of questions, and those wins carry

fewer gold sentences (1.47 against 1.72), so stripping *amplifies* a label-sparsity effect rather than creating a disambiguation one. Corpus-level ambiguity is real - 20.8% of syllables collapse - but we could not link it to per-question loss, and we report that failure (Appendix F.2).

A negative result on hyperparameters. Sweeping $\lambda \in [0.0, 1.0]$ and $k \in [70, 95]$ on VimQA *under a 0.5B reader at* $n = 50$ puts the best λ at 0.4 against the paper's 0.6 - but the gap is 0.8 F1, and resolving that would need $n \approx 37{,}000$ questions against VimQA's 1,003. We report a **negative result**: no evidence the paper's hyperparameters need changing for Vietnamese. On that sweep $k = 90$ is also optimal, matching the paper; the 7B sweeps in Appendices D.2 and D.1 put the best k elsewhere, but every one of those gaps is inside its own detection threshold, so the three readings disagree only within noise.

B Prompts Used

The original paper does not release prompt text. We give ours in full so that subsequent work can reproduce our setup exactly. Placeholders in `{...}` are substituted at run time.

B.1 Answerability judgment (Step 2)

```
You are solving a multi-hop question.
Below is the evidence collected so far.

QUESTION: {question}

EVIDENCE SO FAR:
{evidence}

Is the evidence above SUFFICIENT to
answer the question accurately?
Reply with exactly one word:
ANSWERABLE_YES or ANSWERABLE_NO
```

B.2 Follow-up question generation (Step 3)

```
You are solving a multi-hop question but
the evidence is not yet sufficient.

ORIGINAL QUESTION: {question}

EVIDENCE SO FAR:
{evidence}

State ONE short follow-up question
targeting exactly the MISSING information
```

```
needed to answer the original question.
Give only the follow-up question, no
explanation.

FOLLOW-UP:
```

B.3 Reader prompt

Shared by *every* compression method, so that comparisons differ only in the context supplied. The prompt is bilingual because we evaluate both English and Vietnamese datasets.

```
Answer the question using ONLY the
context below. Reply with the SHORT
answer only (a few words, no
explanation, no full sentence). If the
context does not contain the answer,
reply: unknown

CONTEXT:
{context}

QUESTION: {question}
ANSWER:
```

C Results at Full Scale

C.1 VimQA at $n = 1{,}003$

The tables in the body mix sample sizes - $n = 50$ for the paper comparison, $n = 200$ for the baselines - because each was run when the compute allowed. This appendix gives the largest: all four methods on **all 1,003 VimQA questions** with the same 7B reader, which drops the paired detection threshold from 18.0 F1 to **4.3**.

Method	EM	F1*	ratio
Raw Documents	28.5	46.87	1.000
Oracle	**36.4**	**61.03**	0.057
LLMLingua-2	17.2	35.31	0.316
IterCOMP	26.9	51.29	**0.206**

Three paired comparisons, 10k bootstrap resamples:

Comparison	ΔF1	95% CI
IterC. − LLML-2	+15.98	[+12.98, +19.01]
IterC. − Raw	+4.42	[+1.73, +7.17]
Oracle − IterC.	+9.74	[+7.08, +12.42]

C.2 What the move from $n = 50$ cost MuSiQue

MuSiQue is the one dataset we have at both sizes, so it shows what the small sample was hiding. Oracle falls 57.05 to 49.84, Raw 32.85 to 23.43 and IterCOMP 35.11 to 26.74; only LLMLingua-2 rises, 13.51 to 15.14. The first fifty questions were simply easier than the set. Two of our readings changed with it, both recorded in Section 7: at $n = 500$ IterCOMP no longer separated from Raw on English (later resolved in its favour at $n = 1{,}500$, Appendix C.3), and the margin over LLMLingua-2 is larger on Vietnamese (+15.98) than on English (+11.60), reversing the $n = 50$ reading that had looked like evidence for the advantage growing with benchmark difficulty.

One reading strengthened. The compressor reaches **84%** of the Oracle score on VimQA against **54%** on MuSiQue - wider than the 81% against 62% at $n = 50$. Oracle/Raw moves the same way, 2.13 on MuSiQue against 1.30 on VimQA, consistent with this ratio being set by how many distractor passages a dataset supplies: MuSiQue gives 20 candidates per question, VimQA 10.

C.3 All four datasets at $n \geq 500$

Every comparison with the original rested on MuSiQue until now: 2Wiki and HotpotQA had been run only at $n = 50$ under a 0.5B reader, which cannot be set beside their LLaMA-3-8B table. Both now run at $n = 1{,}500$ with the 7B reader and the paper's own k per dataset - 85 for 2Wiki, 90 for HotpotQA. This appendix expands Table 5 dataset by dataset.

The central claim holds on all four. Sentence-level compression beats token-level compression by +11.60 to +18.58 F1, with every interval clear of zero. That is the paper's thesis, and it is the one result that does not depend on which dataset we look at.

The full ordering does not. The paper reports Oracle > IterCOMP > Raw > LLMLingua-2 throughout. We reproduce it on VimQA (+4.42, clear). On MuSiQue the order matches and its +3.31 margin (crossing zero at the table's $n = 500$) clears at $n = 1{,}500$; on 2Wiki the order matches but +1.00 does not separate. Only on HotpotQA does **Raw overtake IterCOMP**, by **4.66 F1, interval clear**. An earlier version of this paper claimed in its abstract that "the method ordering reproduces exactly"; that claim was made from MuSiQue alone and is withdrawn.

What tripling the sample resolved. At $n = 1{,}500$, MuSiQue's IterCOMP−Raw goes from +3.31 (crossing zero) to +4.19 (clear) and 2Wiki's flips sign, −2.69 to +1.00. Greedy decoding makes the first 500 of the 1,500 reproduce to 0.00 F1, so the $n = 500$ readings were too few, not wrong; we had published that −2.69 needs $n \approx 1{,}300$. Below the threshold of Section 7.1, a difference has only a value, not a reliable sign.

Why, and what it costs the premise. Our reader reads long contexts better than theirs. On HotpotQA Raw scores 59.24 against the paper's 43.63 - +15.6 F1 from the reader alone, the largest such gap in this work - while IterCOMP moves only from 51.78 to 54.58. A reader that copes with distractors gains more from being handed everything than from being handed a filtered subset, so compression's *accuracy* advantage evaporates and only its *cost* advantage survives: IterCOMP still reads 19.4% of the tokens.

This qualifies the premise stated in Section 1, that irrelevant passages degrade reader accuracy. They do on VimQA and MuSiQue, where Oracle beats Raw by 1.30 and 2.13×. On HotpotQA the ratio is 1.21 and on 2Wiki 1.61, and the reader recovers enough from the full context that filtering costs accuracy rather than buying it.

C.4 MuSiQue at matched compression

At $k = 90$ - the paper's setting - our retention on MuSiQue is 0.349 against their 0.14 (Table 10). Same knob, same dataset, 2.5× apart, so the difference is in the segment population the percentile ranks over rather than in the percentile: MuSiQue supplies 20 passages per question and our decomposition yields more candidates than theirs. Comparing F1 across that gap says little, so we swept k to find where our retention meets theirs ($n = 200$, 7B reader; the target is the ratio, measured precisely at any n).

k	keep	F1	iters	segs
90 (paper's)	0.330	26.65	3.14	6.3
93	0.287	26.32	3.19	5.4
96	**0.160**	**29.04**	3.00	3.0
97.5	0.160	29.04	3.00	3.0
paper	*0.14*	*27.36*	–	–

Table 10. MuSiQue, Qwen2.5-7B at 4-bit, $n = 200$. $k = 96$ and $k = 97.5$ are identical because the filter floors at three segments per question.

At $k = 96$ we retain 0.160 against their 0.14 and score **29.04** against **27.36**. So at comparable compression our reimplementation is +1.68 above the original, not 0.62 below - the deficit reported at $k = 90$ was an artefact of reading 2.5× more context, which costs accuracy on MuSiQue because the extra passages are distractors.

Two limits on this. The sweep floors at $k = 96$: raising k to 97.5 changes nothing because only three segments survive, so we cannot reach their 0.14 exactly. And +1.68 is well inside the detection threshold at $n = 200$, so the honest statement is that the two are indistinguishable at matched compression - a stronger reproduction result than either the −0.62 or the +1.68 read alone.

D Ablations and Open Questions

D.1 Ablations at $n = 500$, which refute one of them

We reran the two axes that carry claims, at ten times the sample; the run is released as `ablation_vimqa500_7b.json`.

	n=50		n=500			n=500		
`m_it`	F1	rat.	F1	rat.	it.	k	F1	rat.
1	28.36	.134	31.7	.133	1.00	70	35.9	.431
2	**33.03**	.158	34.9	.160	1.22	80	**38.3**	.320
3	**33.03**	.170	35.6	.174	1.36	85	38.2	.251
4	**33.03**	.181	**36.2**	.185	1.47	90	36.1	.194
5	**33.03**	.189	36.1	.194	1.58	95	38.2	.129

Table 11. VimQA, Qwen2.5-7B at 4-bit; "rat." is the retained fraction, "it." the mean iteration count. At $n = 50$ the F1 column is *bit-identical* from `max_iter` = 2 (33.033695 to six decimals); at $n = 500$ it is not. Detection threshold at $n = 500$ is 6.6 F1 paired, so *no* pair in either half separates.

Two things follow, and the first corrects this appendix.

The bit-identical result was an artefact of $n = 50$. The left half of Table 11 shows how complete the illusion was: extra passes changed which segments were retained (0.158 to 0.189) without changing a single answer, to six decimals. An agreement that exact reads as a structural fact rather than a sampling one, and we inferred that no larger sample would alter those four numbers. A larger sample did: at $n = 500$ they run 34.9, 35.6, 36.2, 36.1, with mean iterations rising 1.18 to 1.58. Fifty questions were simply too few to contain the minority on which the ceiling binds - and that minority is narrow, since even at `max_iter` = 5 the loop stops after 1.58 passes on average. The remaining finding is weaker: two passes versus five yield +1.2 F1, far inside noise, for 20% more context - an argument for the smaller budget on cost, not accuracy.

The k-curve reversal is consistent with sampling noise. At $n = 50$, $k = 70$ scored best (38.77) and $k = 90$ worst (33.03), a spread of 5.7 F1. At $n = 500$, $k = 70$ is *worst* (35.9) and the spread has collapsed to 2.3. We expected the larger sample to resolve this comparison and it did the opposite: the effect shrank rather than the interval. A ranking that inverts under a tenfold sample increase was never a ranking, and the caution this report attached to the $n = 50$ ablations was warranted.

D.2 What this reimplementation does not settle

One question we consider open, and one we have since closed.

Whether the stopping rule helps - resolved. At $n = 50$, removing the answerability check appeared to raise F1 from 33.03 to 36.64 (the `no-answerability` ablation), suggesting the rule *lost* 3.6 F1 - below the 18.0-point paired threshold, so we ran four stopping regimes at $n = 200$ under the 7B reader. A *perfect* oracle stopper scores 49.91 against the paper's 50.08 on VimQA and 25.94 against 25.90 on MuSiQue (-0.17, $+0.04$): the rule incurs no accuracy cost, and the earlier gap was a sampling artifact. Never stopping instead reads 49.7% of the context against 18.1% for the same F1, so the rule's value is *token savings, not accuracy* - a distinction the paper does not draw.

Whether a larger reader lifts the ceiling. Oracle, seeing exactly the gold sentences, still scores zero on 28% of VimQA and 30% of MuSiQue questions; since compression cannot beat Oracle, that fraction bounds what Section 8 attributes to the reader. The 98 age-comparison questions are the identified part; the rest we have not categorised, and whether a reader at the paper's scale recovers either is beyond this GPU budget.

Whether $k = 90$ is the right operating point. Sweeping the percentile threshold on VimQA traces a monotone trade-off that the single published value hides:

k	70	80	85	**90**	95
F1	38.77	36.60	36.35	**33.03**	34.56
ratio	0.430	0.316	0.245	**0.189**	0.118

At $n = 50$ every step from $k = 70$ to $k = 90$ appears to trade accuracy for compression, placing $k = 90$ near the aggressive end (43% of the context against 19% for 5.7 F1). **That trade-off does not survive** $n = 500$ (Appendix D.1), where $k = 70$ becomes the *worst* setting and the spread collapses to 2.3 F1: the monotonicity was a property of fifty questions, and no tested k is distinguishable from another.

Whether two design choices earn their complexity. Equation (4) combines dense and sparse signals, yet on VimQA either half alone scores *above* the combination (lexical 35.44, dense 35.12, dual 33.03), and even BM25 lands within 0.05 F1 at 32.98; top-3 reaches 37.46 F1 at 16.3% of the context against 33.03 at 18.9% for $k = 90$. Both gaps sit inside the noise floor, so neither ablation distinguishes its alternatives at $n = 50$ (separation would need $n \approx 1{,}000$); we report them as absence of evidence *for* either choice, not evidence against. Appendix D.1 moves two axes to $n = 500$ and refutes one; the scorer comparison, its 2.4 F1 spread under threshold even at $n = 500$, is left at $n = 50$.

E Token Fertility: Supplementary Measurements

Section 2 gives the headline fertility numbers. The per-token counts behind them, and the sensitivity of the effective ratio to the tokeniser, are reported there in full; this appendix is retained only as a pointer so that appendix lettering matches the submitted artefact.

F Further Vietnamese Analysis

F.1 Function-word damage

We also tested, and *refuted*, a second metric. Token-level compression was expected to sever syllables and tone marks (*phở* → *ph*), but on 50 VimQA questions LLMLingua-2 severs **zero** syllables; what it cuts is *function words*. At matched compression the function-to-content loss ratio is unbiased for sentence-level IterCOMP but not for token-level LLMLingua-2, and the English control lands in the same place:

n=500	IterC.	LLML-2	gap
VimQA (vi)	1.01×	1.17×	+0.16
MuSiQue (en)	0.99×	1.17×	+0.17

At $n = 50$ the gaps are larger (+0.31, +0.39) but shrink with sample size, confirming the bias is a property of compression granularity, not the language; our typological hypothesis gets no support. The damage is concrete - at `rate` = 0.33 LLMLingua-2 discards 96.1% of relational and 91.2% of negation particles, and dropping *không* inverts a sentence - yet it barely predicts F1: only at $n = 1{,}003$ does the token-level correlation clear zero ($\rho = -0.067$, 95% CI $[-0.124, -0.004]$), a real but negligible association ($|\rho| < 0.1$), while sentence-level IterCOMP stays null. A modern reader recovers a sentence stripped of function words when the evidence is present, but not one that was dropped - so grammatical damage is a measurable property of compressor output, not a demonstrated cause of the 11.6-18.6 F1 gap, whose mechanism remains open.

F.2 Undiacriticised Vietnamese

Section 7.4 reports the headline result and its table (Table 7): with tone marks intact gold context beats IterCOMP by 4.86 F1, but once diacritics are stripped IterCOMP beats gold context by 3.78 to 5.82 F1. We evaluate `strip` (context only) and `both` (context and question) against a gold-context branch at $n = 200$; here we trace the mechanism.

How much ambiguity stripping creates. The likely mechanism is that undiacriticised text introduces lexical ambiguity that wider context resolves, while gold context removes exactly that surrounding material. On the corpus, of the 13,165 distinct lowercase syllables in VimQA, removing tone marks collapses them onto 11,182, so **20.8% become ambiguous** and 1,983 distinctions are lost (against only 2% diacritic-bearing tokens in English MuSiQue).

Not confined to stripped text. The effect predates stripping: with diacritics intact IterCOMP already beats Oracle on **9.8%** of the 1,003 questions (84.7 F1 against 17.7), so stripping amplifies a background effect rather than causing one. It lives in the labels - IterCOMP's wins carry fewer gold sentences (1.47 against 1.72) and are not luck (perfect F1 on 56% of them where Oracle scores zero on 70%). We have *not* shown ambiguity to be the cause - a per-question ambiguity measure saturates because every common function word collides - so we report the effect and the mechanism separately.

F.3 Vietnamese/English decomposition

	Oracle	Or/Raw	IC/Or
MuSiQue (en)	57.0	1.74	0.62
VimQA (vi)	53.3	**1.22**	**0.81**

Table 12. Same reader (Qwen2.5-7B at 4-bit), $n = 50$, F1 after boolean normalisation. Or/Raw is what perfect evidence selection is worth on the data; IC/Or is IterCOMP's score as a fraction of the Oracle bound.

On VimQA the ordering changes (Table 12) - Raw edges out IterCOMP (43.8 against 43.0) - but with the reader fixed IterCOMP reaches 84% of the Oracle ceiling against 54% on MuSiQue. What differs is Oracle/Raw, 1.22 against 1.74: VimQA gives 10 passages per question against MuSiQue's 20, so the gain is the data's, not the language's.